



ADVANTEST CORPORATION

SmartShell 7 System Reference

User's Guide

2020-11-25

Notices

Legal Notices

All rights reserved. All text and figures included in this publication are the exclusive property of Advantest Corporation. Reproduction of this publication in any manner without the written permission of Advantest Corporation is prohibited. Information in this document is subject to change without notice.

Trademarks and Registered Trademarks

- ADVANTEST is a trademark of Advantest Corporation.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

Warranty

The material contained in this document is provided "as is," and is subject to being changed, without notice, in future editions. Further, to the maximum extent permitted by applicable law, Advantest disclaims all warranties, either express or implied, with regard to this material and any information contained herein, including but not limited to the implied warranties of merchantability and fitness for a particular purpose. Advantest shall not be liable for errors or for incidental or consequential damages in connection with the furnishing, use, or performance of this material or of any information contained herein. Should Advantest and the user have a separate written agreement with warranty terms covering the material in this document that conflict with these terms, the warranty terms in the separate agreement shall control.

Technology Licenses

The hardware and/or software described in this document are furnished under a license and may be used or copied only in accordance with the terms of such license.

Restricted Rights Legend

If software is for use in the performance of a U.S. Government prime contract or subcontract, Software is delivered and licensed as "Commercial computer software" as defined in DFAR 252.227-7014 (June 1995), or as a "commercial item" as defined in FAR 2.101(a) or as "Restricted computer software" as defined in FAR 52.227-19 (June 1987) or any equivalent agency regulation or contract clause. Use, duplication or disclosure of Software is subject to Advantest's standard commercial license terms for 93000 systems and non-DOD Departments and Agencies of the U.S. Government will receive no greater than Restricted Rights as defined in FAR 52.227-19(c)(1-2) (June 1987). U.S. Government users will receive no greater than Limited Rights as defined in FAR 52.227-14 (June 1987) or DFAR 252.227-7015 (b)(2) (November 1995), as applicable in any technical data.

Safety Notices

CAUTION

A CAUTION notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in damage to the product or loss of important data. Do not proceed beyond a CAUTION notice until the indicated conditions are fully understood and met.

WARNING

A WARNING notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in death or severe injury. Do not proceed beyond a WARNING notice until the indicated conditions are fully understood and met.

Table of Contents

1	Introduction	4
2	Components in SmartShell Environment	5
2.1	Basic Components in SmartShell Application	5
2.2	Support for Easy Connection in SmartShell Application	5
2.3	Support for EDA Tools.....	6
2.4	SmartShell Client GUI.....	6
3	Getting Started	7
3.1	Installation Package	7
3.2	SmartShell Root Path	7
3.3	SmartShell Server and Configuration	7
4	How to Launch SmartShell Server	9
4.1	SmartShell Server Launch Option	9
4.2	Launch SmartShell Server	9
4.2.1	SmartShell Console Mode.....	9
4.2.2	SmartShell User Client Mode.....	10
4.3	Switching SmartShell Operation Modes and Exiting SmartShell.....	10
4.4	SmartShell Online Help Menu	10
5	User's SmarTest Test Program Template	13
5.1	Test Program for Mentor ATE-Connect.....	13
6	User's SmartShell Client Test Program	15
6.1	Configure Client Setup	15
6.2	SmartShell Client Test Script.....	15
6.2.1	Python Script.....	15
6.3	Config Setup for Mentor ATE-Connect.....	17
7	Running Test via SmartShell	19
7.1	Running Test directly in SmartShell Server.....	19
7.2	Running user's SmartShell Test Program Controlling SmarTest.....	19
7.3	Running EDA Tool directly from EDA Test Script	20
7.3.1	Running Mentor SiliconInsight via Mentor ATE-Connect.....	20

1 Introduction

SmartShell is a standard V93000 scripting interface to enable a test program or a test script other than a conventional V93000 SmartTest test program to control V93000 tester resources, execute the tests and retrieve the execution results. The SmartShell instructions and data are in standardized ASCII format whether they are stimulus or response.

SmartShell supports SmartTest 7 and SmartTest 8. It can link to EDA tools directly. SmartShell for SmartTest 7 supports digital pattern bring-up. SmartShell for SmartTest 8 supports both digital pattern bring-up and all other applications available on V93000 hardware. The command sets of SmartShell for SmartTest 7 and SmartTest 8 are nearly identical.

Note:

In this document, we may use "test script" interchangeably with the words "test program" other than a conventional V93000 SmartTest test program to control V93000 SOC tester in the context of the SmartShell application.

2 Components in SmartShell Environment

2.1 Basic Components in SmartShell Application

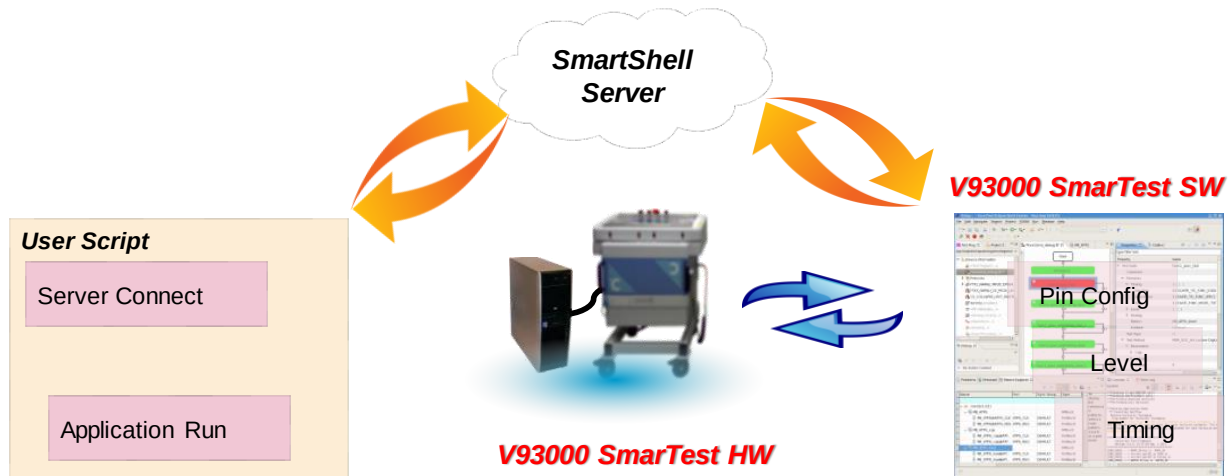


Fig. 1: Basic Components in SmartShell Environment

The basic components to run a SmartShell application in the SmartShell environment are:

- 1) An existing and ready-to-run SmarTest test program
 - a. pin config, level, timing, etc.
 - b. continuity, leakage, power short test
 - c. one or several functional tests for an integrity check of the device
- 2) A client test program or test script using SmartShell commands and this client test program is called a SmartShell client test program.

2.2 Support for Easy Connection in SmartShell Application

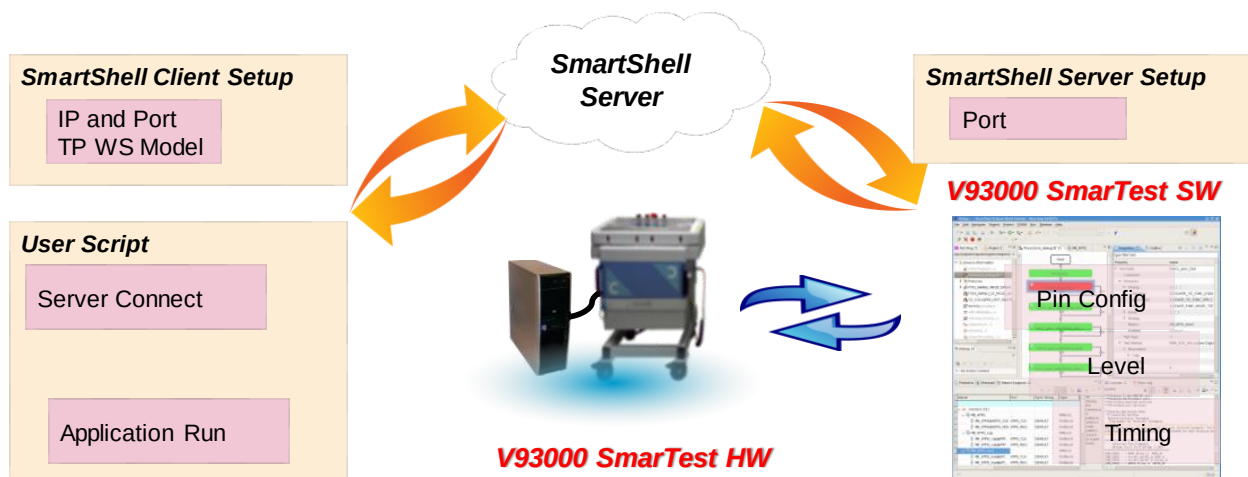


Fig. 2: Components in Enhanced SmartShell Environment

To make the connection easier, a SmartShell client test program can utilize a configuration file: the SmartShell client configuration setup file `smartshell_config.xml`. See details in the next chapter.

2.3 Support for EDA Tools

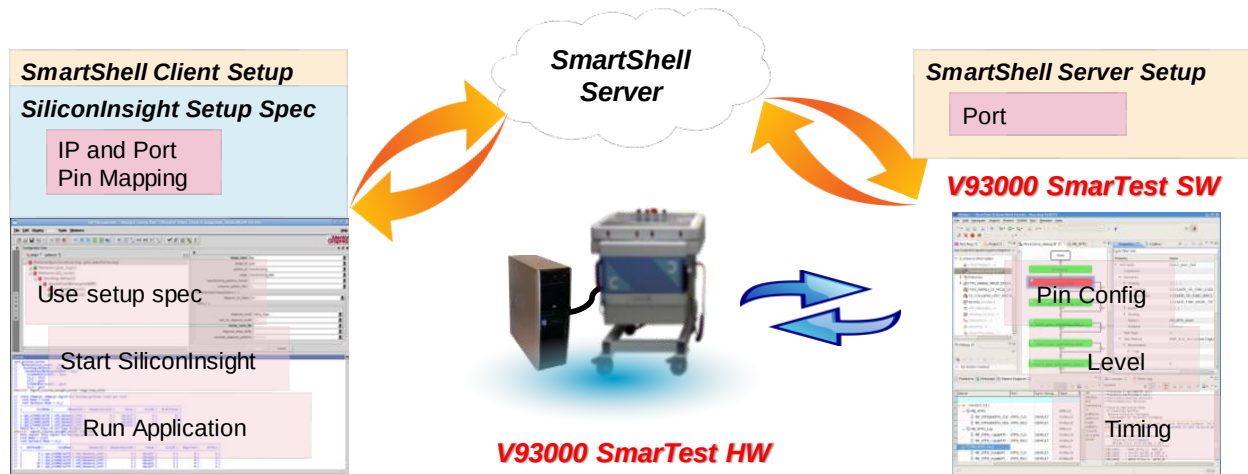


Fig. 3: Components in Mentor Tessent SiliconInsight Environment

Beyond native SmartShell application, SmartShell also supports EDA tools such as Mentor Tessent SiliconInsight and Synopsys TestMax. The connection to Tessent SiliconInsight is established via Mentor ATE-Connect. The connection to TestMax is established via Synopsys ALE. In this way designer's program can be run directly from the EDA environment via SmartShell on a V93000 SOC tester.

Note:

For Mentor, the interface supports only JTAG application and pattern is SVF or Serial Vector Format via the JTAG interface.

2.4 SmartShell Client GUI

SmartShell delivers a SmartShell Client GUI which can perform the following:

- to connect to an active SmartShell server running on a V93000 SmarTest tester workstation;
- to interactively control SmarTest with the SmartShell command set;
- to manage multiple test tasks of any programming language for execution in a queue;
- to convert patterns in any pattern format by using a customer-specified conversion tool;
- to add setups incrementally and dynamically to the SmarTest system for a test execution;
- to perform comprehensive shmoo for an entire testflow or testsuites or pattern.

For details, please refer to the SmartShell reference document: *"SmartShell for SmarTest 7 - SmartShell GUI"*.

3 Getting Started

3.1 Installation Package

SmartShell for SmarTest 7 is a standalone package:

```
v93000_smartshell-x.x.x.x-el#.x86_64.rpm
```

where # stands for 5 (RHEL5) or 7 (EHEL7).

3.2 SmartShell Root Path

The SmartShell software installation folder is in

```
/opt/hp93000/smartshell64-x.x.x.x_el#
```

```
/opt/hp93000/smartshell -> /opt/hp93000/smartshell64-x.x.x.x_el#
```

3.3 SmartShell Server and Configuration

The SmartShell server and the SmarTest run on the same workstation. The SmartShell server will get the access to a SmarTest session in the following way:

- 1) Launch SmarTest
- 2) Load test program
- 3) Launch SmartShell server.

Upon the launch of the SmartShell server, the SmartShell server will look up setups in the configuration setup file `config.xml` as shown in Fig. 4. After that, the SmartShell server waits for a client to connect to it for the SmartShell operation.

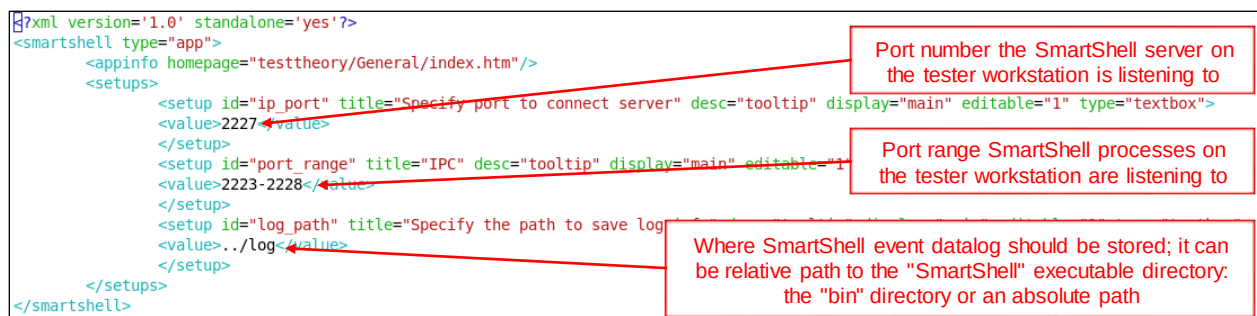


Fig. 4: SmartShell System "config.xml" File

This `config.xml` file can be edited. It can be found in:

```
/opt/hp93000/soc/smartshell/config
```

It is a symbolic link to the following folder:

```
/var/opt/hp93000/soc_common/smartshell/config/
```

The `ip_port` of 2227 is the default main port between SmartShell server and user's SmartShell client test program. The `log` folder stores temporary files which will be deleted upon the closing of the SmartShell server. Only the log folder location can be changed.

4 How to Launch SmartShell Server

4.1 SmartShell Server Launch Option

The level of log details can be specified when SmartShell server is launched:

```
/opt/hp93000/smartshell/bin/SmartShell <log options>
```

Log options are:

logcommand: log commands

logresult: log results

logprint: log both commands and results

All options can be used and be separated by blank.

The log file which logs the commands and the executions results is named as:

```
SmartShellEvent.log
```

When the software crashes, the last 100 commands are logged in a log file named as:

```
SmartShellDump.log
```

They are stored under:

```
/opt/hp93000/smartshell/log
```

This folder is a link to:

```
/var/opt/hp93000/soc_common/smartshell/log
```

Older log files are renamed as:

```
<logfile>_YYYY-MM-DD.HH:MM:ss
```

4.2 Launch SmartShell Server

There are two modes when launching SmartShell Server:

- 1) SmartShell console mode
- 2) SmartShell user client mode.

4.2.1 SmartShell Console Mode

In this mode, the user can type commands interactively in the SmartShell server console window at a command-prompt "SERVER > ". To launch SmartShell server in this mode, use the `scm` option:

```
/opt/hp93000/smartshell/bin/SmartShell scm <log options>
```

```

=====
==== SmartShell v3.4.4.4 - 20200305 ====
(c) 2017, Advantest Co., R&D Shanghai
=====
Connect to SmarTest session :1
Analyzing testflow A1_FT1...
Analyzing pin configuration ADV.cfg...
Analyzing timing A1.tim.mfh...
Initialization has succeeded
HINT: to break server mode, type "break":
SERVER >

```

Fig. 5: SmartShell Console Mode for Interactive Operation

4.2.2 SmartShell User Client Mode

In this mode, user's SmartShell client test program will connect to the SmartShell server and execute the tests. To launch the SmartShell server in this mode:

```
/opt/hp93000/smartshell/bin/SmartShell <log options>
```

```

=====
==== SmartShell v3.4.4.4 - 20200305 ====
(c) 2017, Advantest Co., R&D Shanghai
=====
Connect to SmarTest session :1
Analyzing testflow A1_FT1...
Analyzing pin configuration ADV.cfg...
Analyzing timing A1.tim.mfh...
Initialization has succeeded
#Waiting for client to connect...

```

Fig. 6: SmartShell User Client Mode

4.3 Switching SmartShell Operation Modes and Exiting SmartShell

To switch from the SmartShell user client mode to the SmartShell console mode, execute the following command from the command line in a Unix shell console window:

```
/opt/hp93000/soc/smartshell/bin/to_server_mode
```

To switch from the SmartShell console mode to the SmartShell user client mode, enter a command `break` at the server prompt `SERVER >`

```
SERVER > break
```

To quit SmartShell, either press `Ctrl+C` in the SmartShell console window `"SERVER > "` or enter `quit` or `exit`, or execute `/opt/hp93000/soc/smartshell/bin/killsms` command line in a Unix console window.

4.4 SmartShell Online Help Menu

In the SmartShell server console window regardless whether the SmartShell server was launched in the SmartShell Console Mode or in the SmartShell User Client Mode, as soon as a SmartShell prompt `"SERVER > "` appears, the user can directly issue SmartShell commands. In the SmartShell User Client Mode, the SmartShell prompt `"SERVER > "` appears only during the execution of a user's SmartShell

client test program. To learn the SmartShell commands, the user can use the SmartShell online help menu and type HELP

```
SERVER > HELP
```

```
SERVER > HELP
HELP
CHK          CLOSE_SMT  COPY          DELETE        EDIT          ENABLE        EXE          GET
HELP        HOLD       INIT          INSERT        SET           SHELL        SPLIT       USE
```

Fig. 7: Online Help Menu to Get List of Commands

The important commands are SET, GET, and EXE commands and if the user wants to know commands starting with GET, then issue a help command like this:

```
SERVER > HELP GET
```

It will show all possible matched commands.

```
SERVER > HELP GET
HELP GET
GET ALL_SESSIONS:      GET ALL_SESSIONS
GET EDGE:              GET EDGE <name>+
GET PAULC_TAG:         GET [PAULC] [PAULC_ONF] [PAULC_PEN] [TAG] [-<minlist>] [STARTC=<sgen>] [NO_OF_CYCLES=<=>]
GET PAULC:              GET [PAULC] [PAULC_ONF] [PAULC_PEN] [-<minlist>] [STARTC=<sgen>] [NO_OF_CYCLES=<=>]
GET FLOW_SUITE_LIST:   GET FLOW [-<flow>] SUITE_LIST
GET LEV:               GET LEV <name>+
GET PAT PATH:         GET PAT PATH
GET PAT PATH=<=>:       GET PAT PATH=<=>
GET PAT PRGMRY:        GET PAT PRGMRY
GET PAT SIGNAL_LIST:   GET PAT <label> SIGNAL_LIST
GET PAT TOTAL_CYC:     GET PAT <label> TOTAL_CYC
GET PAT TOTAL_VEC:     GET PAT <label> TOTAL_VEC
GET PAT SEC_SIGNAL:    GET PAT <label> SEC <name> <start> <stop>
GET PAT SEC:           GET PAT <label> SEC <start>
GET PAT SEC_REPEAT:    GET PAT <label> SEC_REPEAT <sec_num>
GET PAT [-<=>] LABEL_LIST: GET PAT [-<=>] LABEL_LIST
```

Fig. 8: Online Help Menu to Get List of Commands Starting with GET

The user wants to learn how to use the command GET PAT TOTAL_VEC:

```
SERVER > HELP GET PAT TOTAL_VEC
```

```
SERVER > HELP GET PAT TOTAL_VEC
HELP GET PAT TOTAL_VEC
NAME:
    GET PAT TOTAL_VEC

DESCRIPTION:
    The "GET PAT TOTAL_VEC" command gets total vector number in a given main label

SYNTAX:
    GET PAT <label> TOTAL_VEC

PARAMETERS:
    label : main label name

USAGE:
    GET PAT jtag_pattern TOTAL_VEC

RETURN:
    total vector number
```

Fig. 9: Online Help Menu to Show the Command Syntax

If the user types an unknown command, the interface will show a SYNTAX ERROR message and at the same time lists all matched commands. For example, if the user typed GET PAT, since it is incomplete, the interface lists all commands starting with GET PAT.

```

SERVER > GET PAT
ERROR: SYNTAX ERROR: command "GET PAT" correct syntax is below
GET PAT LABEL_LIST MAIN_LABELS: GET PAT LABEL_LIST MAIN_LABELS [<port>]
GET PAT LABEL_LIST MULTI_PORT: GET PAT [<mpbburst>] LABEL_LIST MULTI_PORT
GET PAT LABEL_LIST SINGLE_PORT: GET PAT LABEL_LIST SINGLE_PORT
GET PAT LABEL_LIST: GET PAT <label> LABEL_LIST <port>
GET PAT PORT_NAME: GET PAT <label> PORT_NAME <pin>
GET PAT PRIMARY: GET PAT PRIMARY
GET PAT SIGNAL_LIST: GET PAT <label> SIGNAL_LIST
GET PAT TOTAL_CYC: GET PAT <label> TOTAL_CYC
GET PAT TOTAL_VEC: GET PAT <label> TOTAL_VEC
GET PAT VEC SIGNAL: GET PAT <label> VEC <pin> <start> <stop>
GET PAT VEC: GET PAT <label> VEC <start> [WAVEFORM_TABLE]

```

Fig. 10: Online Help Menu to Get List of Commands

In this way, the user can also find it out what is available and learn how each of these commands works.

In addition, entering a SmartShell command behaves similar like a normal text editor and was named as the "presskey" command editor, e.g. arrow up and down key will display entered command before and after the current one, Ctrl-r and Ctrl-s will search a command in the command history and an entered command can be re-used. Entering a command is also like editing a text, e.g. single ESC strike and then click Backspace key will delete the word before until it hits special tokens, double ESC strikes and then Backspace will delete word before the current cursor until blank or other special tokens. Ctrl-z or Ctrl-u are for undo and so on. To see all these functions, enter `HELP SHELL` at the server prompt `SERVER >` as shown in Fig. 11.

DESCRIPTION:
The "SMARTSHELL" interactive interface can issue SmartShell native command directly without writing user script. It enables ease of use in the SmartShell native scripting environment. User can learn first how the command behaves. The commands can be used in any script language.

PARAMETERS:

Command History	
(^)/(v) arrow key:	get existing input; or cursor move up/down for multiple line input
ctrl-p	: get existing input the same as (^) arrow keys
ctrl-n	: get existing input the same as (v) arrow keys
PGUP/PGDN keys	: get existing input up and down
ctrl-r	: get existing input with reverse search of existing inputs
ctrl-s	: get existing input with forward search of existing inputs
Cursor	
(^)/(v) arrow key:	cursor move up/down for multiple line input; or get existing input
ctrl-a	: cursor move to line begin, or at begin of the same line
ctrl-e	: cursor move to line end; or at end of the same line
home	: cursor move to begin of the whole input: first char of first line
end	: cursor move to end of the whole input: last char of the last line
ctrl-f or (>)	: cursor move one char to the right
ctrl-b or (<)	: cursor move one char to the left
ctrl-> arrow	: cursor move one word to the right
ctrl-< arrow	: cursor move one word to the left
Insert	
alt-insert	: insert new line
Delete	
ctrl-d:	: delete this char
alt-d:	: delete previous char
ctrl-h:	: delete previous char
ctrl-k	: delete rest of string input to the left
alt-k:	: delete rest of string input to the right
ctrl-w	: delete word to the right
alt-w	: delete word to the left
esc backspace	: delete word to the left
esc esc backspace:	delete several words to the left
ctrl-c/ctrl-l	: delete the whole input line
Undo	
ctrl-z/ctrl-u	: undo typing
ctrl-y	: redo typing
Update window size	
ctrl-o	: update terminal size when console window size is changed
Server Console Mode	
alt-q	: quit SmartShell
alt-c	: switch to client script mode
User Client Mode	
ctrl-c	: quit SmartShell

Fig. 11: SmartShell Console "presskey" Editor

5 User's SmartTest Test Program Template

A V93000 SmartTest test program must be available or be prepared which contains pin configuration, levels and timing specifications. The basic test program should have a continuity, leakage, power short test to ensure that the board and device are the ones match the loaded V93000 test program and the device operates properly.

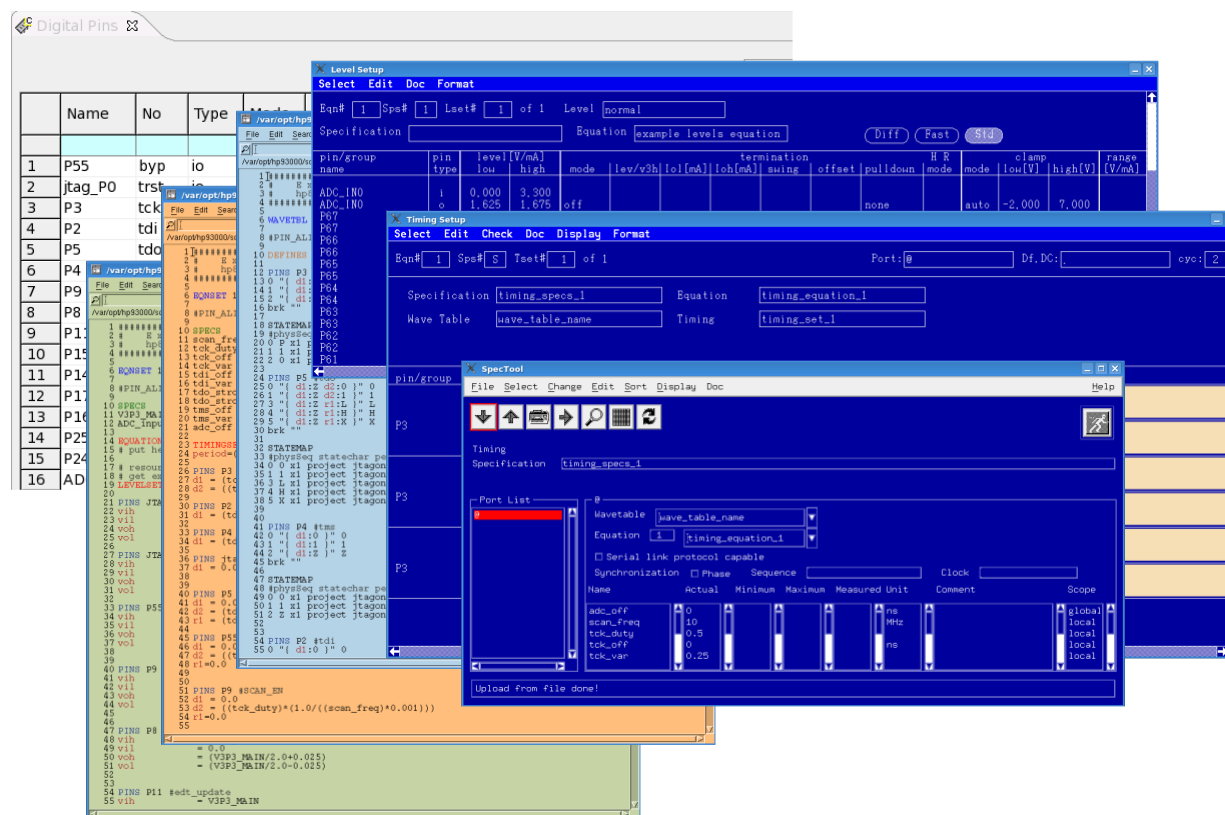


Fig. 12: User's SmartTest Test Program for SmartShell Operation

As soon as a correspondent SmartTest test program for the device under investigation is ready, the SmartShell operation can start.

5.1 Test Program for Mentor ATE-Connect

The primary setting for the Mentor related tests must use a multiport timing specification. This multiport timing specification must define a wavetable with a port containing only these five JTAG pins: tdo, tdi, tck, tms, trst pins. The actual name can defer. The following (Fig. 1) is an example how the wavetable definition should look like. Please note the clock definition in the wavetable for tck needs to have a state char of "P". Not shown in the figure is the wavetable definition for other ports such as test_port and adc_port but these are not mandatory and can also be named arbitrarily.

The testsuite which uses the multiport timing specification can be placed as the last testsuite in the flow. In this way when the flow is executed by SmartShell server, no need to change the primary timing.

```

WAVETBL "wave_table_name" # This starts a wave table defintion
DEFINES jtag_port test_port adc_port
PINS P3 #tck
0 "{ d1:1 d2:F0N }" P
1 "{ d1:1 }" 1
2 "{ d1:0 }" 0
brk ""
STATEMAP
#physSeq statechar period
0 P x1
1 1 x1
2 0 x1

PINS P5 #tdo
0 "{ d1:Z d2:0 }" 0
1 "{ d1:Z d2:1 }" 1
3 "{ d1:Z r1:L }" L
4 "{ d1:Z r1:H }" H
5 "{ d1:Z r1:X }" X
brk ""
STATEMAP
#physSeq statechar period
0 0 x1
1 1 x1
3 L x1
4 H x1
5 X x1

PINS P4 #tms
0 "{ d1:0 }" 0
1 "{ d1:1 }" 1
2 "{ d1:Z }" Z
brk ""
STATEMAP
#physSeq statechar period
0 0 x1
1 1 x1
2 Z x1

PINS P2 #tdi
0 "{ d1:0 }" 0
1 "{ d1:1 }" 1
2 "{ d1:Z }" Z
brk ""
STATEMAP
#physSeq statechar period
0 0 x1
1 1 x1
2 Z x1

PINS jtag_P0 #trst
0 "{ d1:0 }" 0
1 "{ d1:1 }" 1
2 "{ d1:Z }" Z
brk ""
STATEMAP
#physSeq statechar period
0 0 x1
1 1 x1
2 Z x1

```

Fig. 13: Timing Wavetable Example

In addition, user can also specify the timing specification with the SmartShell command "USE" which can be found in the command reference manual. Information on how to configure the interface and how to run the test program with ATE-Connect can be found in section **6.3 Config Setup for Mentor ATE-Connect** and **7.3 Running EDA Tool directly from EDA Test Script** respectively.

6 User's SmartShell Client Test Program

The SmartShell package includes a Python library (for Python 2.7 and 3.7) for developing a user's SmartShell client test program in Python. They are in:

/opt/hp93000/smartshell/lib/python2.7

/opt/hp93000/smartshell/lib/python3.7

6.1 Configure Client Setup

For Python scripting language, the user's SmartShell client configuration file is `smartshell_config.xml` (Fig. 14) and it must be found in the user's script directory. If `host` name is commented out, it will assume that the user script will run from the same tester workstation where the SmartShell server is running.

```
<?xml version="1.0" encoding="UTF-8"?>
<smartshell_config>
  <!--if both ip and host is not defined, 127.0.0.1 is used as default ip.-->
  <host>10.150.110.182</host>
  <!--<host>sjclxmallard</host>-->
  <serverport>2227</serverport>
</smartshell_config>
```

Fig. 14: User "smartshell_config.xml" client setup file for controlling SmartTest externally

6.2 SmartShell Client Test Script

6.2.1 Python Script

In order to establish the link between user script and SmartShell server, the user needs to add codes in the client Python test script to automatically initialize a SmartShell interface instance `shell_interface` between the client script and the SmartShell server. There are two cases how to add the initialization code from SmartShell Python libraries into the client Python test script:

- a) **When the client Python test script is located at the same tester workstation running SmartTest software.**

```
If using the default smartshell_config.xml, add this line to the python script
sys.path.append('/opt/hp93000/smartshell/lib/python2.7')

If using a different smartshell_config.xml, add the following lines to the python script
sys.path.append('/opt/hp93000/smartshell/lib/python2.7')
from spl_imports import *
import spl_globals
spl_globals.init('smartshell_config_online.xml')
```

- b) **When the client Python test script is located at a different computer as the tester workstation running SmartTest software:**

Copy /opt/hp93000/soc/smartshell/lib/python2.7 or /opt/hp93000/soc/smartshell/lib/python3.7 if the user is using the python interpreter revision 2.7 or 3.7 respectively to the folder of the PC or the workstation where the client Python script is located, e.g.

```
> scp <useraccount>@<smartshellhostname>:/opt/hp93000/soc/smartshell/lib/python2.7  
<smartshell python library folder>
```

The client Python test script will need to add the following lines of code:

```
sys.path.append('<smartshell python library>/lib/python2.7')  
This will take the default smartshell_config.xml found in the user's Python script  
folder.  
  
sys.path.append('<smartshell python library>/lib/python2.7')  
from spl_imports import *  
import spl_globals  
spl_globals.init('smartshell_config_online.xml')
```

The user's SmartShell client test program will read the user client configuration file smartshell_config.xml or a different configuration file specified by the line of code spl_globals.init(<smartshell config file>) to connect to the SmartShell server.

To send the SmartShell commands to the server, use the following function

```
shell_interface.sendCommand(<cmd string>)
```

Another python function which has been introduced earlier is still available and does the same:

```
shell_interface.generic_call(<cmd string>)
```

To get results, use the following line of code:

```
shell_interface.getResult()
```

Or to get the result right way:

```
test_results=shell_interface.sendCommand(<cmd string>)  
test_results=shell_interface.generic_call(<cmd string>)
```

6.3 Config Setup for Mentor ATE-Connect

Working with Mentor SiliconInsight, the user only needs to adjust the following entries in the Mentor SiliconInsight EDA environment before running the client test program:

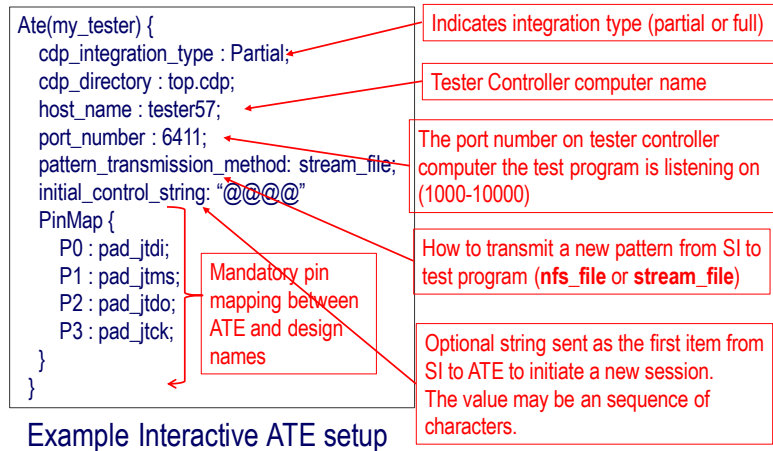


Fig. 15: Mentor SiliconInsight Configuration File "SiliconInsightSetupSpec.cfg"

The user must define a correct pin-to-pin mapping. Fig. 16 shows a more complete setup specification file and how it is configured.

Only when the setup specification file is set up correctly (the correct name of tester workstation, the correct port of socket communication, and the correct pin mapping between design pins and SmartTest test program pins), then the execution of the EDA will work.

Mentor Tessent SiliconInsight software via ATE-Connect will take care of the following:

- 1) Creates test patterns based on the design rules specified by design engineers
- 2) Sends the pattern to SmartShell
- 3) Sends the measurement condition such as frequency and supply voltage to SmartShell
- 4) Sends the pattern modification instruction in the test pattern to SmartShell
- 5) Receives the test results from SmartShell
- 6) Processes the test results automatically within SiliconInsight software
- 7) Repeats this process by user's request or due to an exit condition defined by design engineers

For details on Mentor Tessent SiliconInsight, the user can refer to the Tessent SiliconInsight User's Manual for Tessent Shell.

```

SiliconInsightSetupSpecification {
    selected_setup : Ate(partially_integrated);
    Protocol {
        SvftapPins {
            TRST : trst;
            TDI  : tdi;
            TMS  : tms;
            TDO  : tdo;
            TCK  : tck;
        }
    }
    Ate(partially_integrated) {
        cdp_integration_type : Partial;
        cdp_directory       : interactive_ate.cdp;
        overwrite_cdp_on_startup : on;
        host_name           : atetester;
        port_number         : 2227;
        pattern_transmission_method : stream_file;
    }
    PinMap {
        P0 : trst;
        P3 : tck;
        P2 : tdi;
        P5 : tdo;
        P4 : tms;
        P9 : SCAN_EN;
        P8 : edt_clock;
        P11 : edt_update;
        P15 : edt_channels_in1;
        P14 : test_en;
        P17 : SHIFT_CLOCK;
        P16 : edt_channels_out1;
        P25 : edt_low_power_shift_en;
        P24 : edt_single_bypass_chain;
        P55 : edt_bypass;
        P56 : scan_reset;
        P57 : TRESET;
        P58 : scan_set;
        P59 : RESET;
    }
}

```

JTAG protocol pin Mapping

ATE-Connect to SmartShell Setup

ATE Pins

Design Pins

Design pin to ATE pin Mapping

Fig. 16: Mentor SiliconInsight Configuration File "SiliconInsightSetupSpec.cfg" example

7 Running Test via SmartShell

When there is a user's SmartTest test program available, the user can run the test through SmartShell.

7.1 Running Test directly in SmartShell Server

The user starts the SmartShell server in SmartShell console mode. In the SmartShell console window, the user directly enters commands to control the SmartTest for the SmartShell operation.

```
=====
==== SmartShell v3.4.4.4 - 20200305 ====
(c) 2017, Advantest Co., R&D Shanghai
=====

Connect to SmartTest session :1
Analyzing testflow A1_FT1...
Analyzing pin configuration ADV.cfg...
Analyzing timing A1.tim.mfh...
Initialization has succeeded
HINT: to break server mode, type "break":
SERVER >
```

Fig. 17: Starting SmartShell server and launch user's SmartTest test program

Fig. 17 shows the EXE FLOW command to run the testflow.

```
EXE FLOW
```

Return

```
MG_Test_1 P
MG_Test P
bin_disconnect P
```

7.2 Running user's SmartShell Test Program Controlling SmartTest

Having followed the steps described in **5 User's SmartTest Test Program Template** and in **6 User's SmartShell Client Test Program**, the user can now run the user's SmartShell test program written in Python.

- 1) Start the SmartShell in SmartShell User Client Mode

```
=====
==== SmartShell v3.4.4.4 - 20200305 ====
(c) 2017, Advantest Co., R&D Shanghai
=====

Connect to SmartTest session :1
Analyzing testflow A1_FT1...
Analyzing pin configuration ADV.cfg...
Analyzing timing A1.tim.mfh...
Initialization has succeeded
#Waiting for client to connect...
```

Fig. 18: Launch SmartShell Server

- 2) Run the Python script which will automatically connect to the SmartShell server and execute the tests.

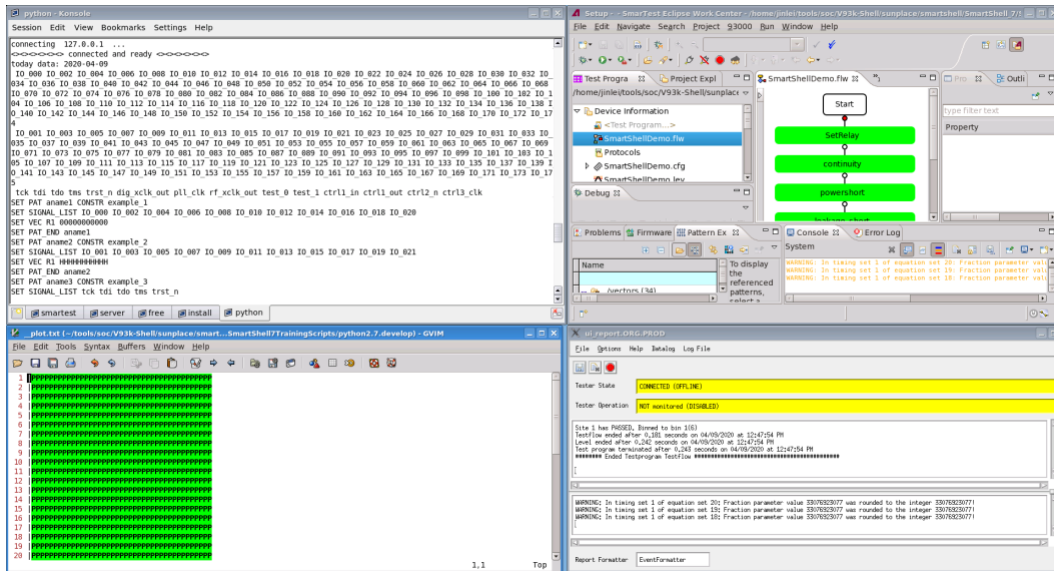


Fig. 19: Launch user's SmartShell Python test script

7.3 Running EDA Tool directly from EDA Test Script

7.3.1 Running Mentor SiliconInsight via Mentor ATE-Connect

Having followed the steps in section 5 *User's SmartTest Test Program Template*, and section 6.1 *Configure Client Setup*, and especially section 6.3 *Config Setup for Mentor ATE-Connect*, the user should be able to run the Mentor SiliconInsight EDA script directly.

- 1) Run SmartShell server as usual;

```
=====
==== SmartShell v3.4.4.4 - 20200305 ====
(c) 2017, Advantest Co., R&D Shanghai
=====

Connect to SmartTest session :1
Analyzing testflow A1_FT1...
Analyzing pin configuration ADV.cfg...
Analyzing timing A1.tim.mfh...
Initialization has succeeded
#Waiting for client to connect...
```

Fig. 20: Launch SmartShell Server

SmartShell for SmarTest 7 - User's Guide 3.4.4.8

